

13

Query Tuning, Indexes, and Performance Optimization

Performance tuning is one of the most complex tasks in the daily job of a **database administrator (DBA)**. SQL is a declarative language, and therefore it does not define how to access the underlying data – that responsibility is left to the database engine. PostgreSQL, therefore, must select, for every statement, the best available access to the data.

A particular component, the planner, is responsible for deciding on the best out of all the available paths to the underlying data, while another component, the optimizer, is responsible for executing the statement with such a particular access plan.

The aim of this chapter is to teach you how PostgreSQL executes a query, how the planner computes the best execution plan, and how you can help in improving the performance by means of indexes.

You will learn about the following topics in this chapter:

- Execution of a statement
- Indexes
- The EXPLAIN statement
- An example of query tuning
- ANALYZE and how to update statistics
- Auto-explain

Technical requirements

You need to know the following:

- How to execute queries against the database
- How to execute **data description language (DDL)** statements

The chapter examples can be run on the `chapter_13` image that you can find in the book's GitHub repository: <https://github.com/PacktPublishing/Learn-PostgreSQL-Second-Edition>. For installation and usage of the Docker images available for this book, please refer to the instructions provided in *Chapter 1, Introduction to PostgreSQL*.

Execution of a statement

SQL is a declarative language: you ask the database to execute something on the data it contains, but you do not specify how the database is supposed to complete the SQL statement. For instance, when you ask to get back some data, you execute a `SELECT` statement, but you only provide the clauses that specify which subset of data you need, not how the database is supposed to pull the data from its persistent storage. You have to trust the database – in particular, PostgreSQL – to be able to do its job and get you the fastest path to the data, always, under any circumstance of workload. The good news is that PostgreSQL is really good at doing this and is able to understand (and to some extent, interpret) your SQL statements and its current workload to provide you with access to the data in the fastest way.

However, finding the fastest path to the data often requires an equilibrium between searching for the absolute fastest path and the time spent in reasoning about this path; in other words, PostgreSQL sometimes chooses a compromise to get you data in a fast-enough way, even if that is not the absolute fastest one.

Sometimes, on the other hand, PostgreSQL cannot understand very well how to find the fastest path to the data, and a DBA can help improve performance. Usually, adding an index can help PostgreSQL retrieve the underlying data in a faster way. Other times, a slow statement hides a miswritten query (i.e., a statement written with incorrect or contradictory clauses). Moreover, slow queries can be due to PostgreSQL reasoning about the wrong size of the dataset it has to handle. In all these cases, the DBA has to provide some tuning in the database or the statements to help PostgreSQL make the best decisions.

In order to be able to help your cluster optimize your statements, you need to first understand how PostgreSQL handles a SQL statement. In the following section, you will learn all the fundamentals of how a SQL statement is converted into a set of actions that PostgreSQL executes to manage data.

Execution stages

A SQL statement – a query, for short – is handled in four main stages:

1. The first stage is parsing; a dedicated component, the parser, handles the textual form of the statement (the SQL text) and verifies whether it is correct or not. If the statement has any syntax errors, the execution stops at this early stage; otherwise, the parser disassembles the statement into its main part, for example, the list of involved tables and columns, the clauses to filter data, sorting, and so on.
2. Once the parser has completed successfully, the statement goes to the second stage: the rewriting phase. The rewriter is responsible for applying any syntactic rules to rewrite the original SQL statement into what will be effectively executed. In particular, the rewriter is responsible for applying rules (refer to *Chapter 8, Triggers and Rules*). When the rewriter has completed its task, producing the effective statement that the database is going to handle, this statement passes to the next stage: optimization.
3. In the optimization phase, the query is handled by the optimizer, which is responsible for finding the fastest path to the data. Finding this fastest path is not a simple task: the optimizer must decide how, from among all the available access methods, such as indexes or direct access, to get to the data. As you can imagine, reasoning and iterating among all the available access methods consumes time and resources, so the task of the optimizer is not only to find out the fastest access method but also to find it out in a short time.
4. Lastly, when the optimizer has decided how to access the data, the query goes to the last phase: execution. The execution phase is handled by the executor component, which is responsible for effectively going to the storage and retrieving (or inserting) the data using the access method decided by the executor.

To summarize, a single SQL statement goes through four stages, all shown in the following diagram: a parsing phase that checks the syntax of the statement, a rewriting phase that transforms the query into something more specific, an optimization phase that decides how to access the data requested by the query, and lastly, an execution phase, which gets physical access to the data.

This can be visualized in the following diagram:

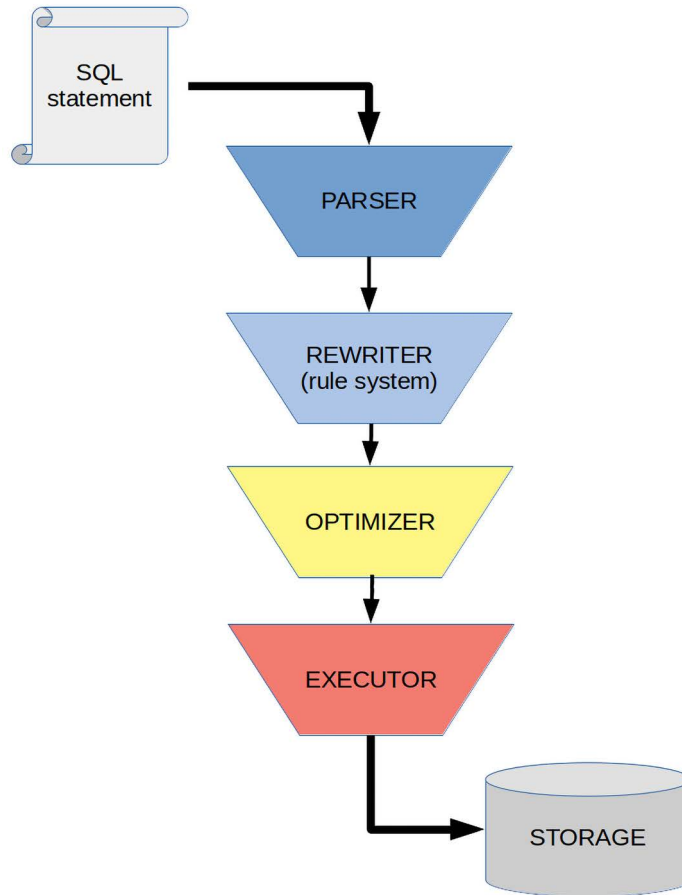


Figure 13.1: PostgreSQL query stages

The DBA can only interact with the database in the optimization phase, trying to help PostgreSQL better understand the statement and optimize it correctly whenever PostgreSQL is not doing an optimal job. The following section takes a closer look at the optimizer, in order to prepare you for the ways you can tune your queries and your database to handle queries in a smarter and faster way.

The optimizer

The optimizer is the component responsible for deciding what to use to access the data as quickly as possible. If a table does not provide any index, then there is only one way to access its data, and so there is nothing to reason about the way to extract data from the table.

On the other hand, if a table provides a few indexes, the optimizer has to decide which one best fits the statement to be executed. The situation becomes much more complex if there are multiple tables, each one with multiple indexes: the optimizer has to reason about all the possible ways to get to the final result.

How can the optimizer choose among the different ways to access the data? The optimizer uses the concept of cost: every way to access the data is assigned a cost and the way that has the lowest cost wins and is chosen as the best access method.

It is for this reason that the PostgreSQL optimizer is called a cost-based optimizer.

PostgreSQL is configured to assign a specific cost to every operation it performs: seeking data from the storage, performing some CPU-based operation (for example, sorting in memory), and so on. The optimizer iterates over all the possible ways of accessing data and mangling it to return to the user the desired result, computing the total amount of cost for every way – that is, the sum of the costs of every operation PostgreSQL will perform. After this, the plan with the lowest cost is passed to the executor as a sequence of actions to perform, and thus data is managed.

This is only half of the story, though. There are cases where the job of the optimizer is really simple: if there is only an access method, it is trivial to decide how to access data. However, there are statements that involve so many objects and tables that iterating over all the possibilities would require a lot of time, so much time that the result will be overtaken by the time spent in computing the optimal way to access the data. For this reason, if the statement involves more than 12 table joins, the optimizer does not iterate all the possibilities but rather executes a genetic algorithm to find a compromise way to access the data. The compromise is between the time spent in computing the path to the data and finding a not-too-bad access path.

The executor can also perform data access using parallel jobs. This means, for instance, that retrieving a very large set of data can be performed by dividing the amount of work between different parallel workers (for example, threads), each one assigned to a smaller subset of the data.

In all the cases, the optimizer divides the set of actions to pass to the executor in nodes; a node is an action to execute in order to provide the final or an intermediate result. For example, say you execute a generic query asking for data in a specific order, as follows:

```
SELECT * FROM categories ORDER BY description;
```

The optimizer will pass two actions to the executor, and thus the nodes: one to retrieve all the data and one to sort the data.

In the following subsections, we will present the main nodes that the optimizer considers and passes to the executor. We will start from the sequential nodes – those nodes that will be executed with a single job – and then we will see how PostgreSQL builds parallelism on top of them.

Nodes that the optimizer uses

In this section, we will present the main nodes you can encounter in the optimizer plan. There are different nodes for every operation that can be performed, and for every different access method that PostgreSQL accepts.

It is important to note that nodes are stackable: the output of a node can be used as the input to another node. This allows the construction of very complex execution plans made by different nodes, which can produce a fine-grained access method to the data.

Sequential nodes

Sequential nodes are those nodes that will be executed sequentially, one after the other, in order to achieve the final result. The main nodes are listed here and will be explained in the following subsections:

- Sequential Scan
- Index Scan, Index-Only Scan, and Bitmap Index Scan
- Nested Loop, Hash Join, and Merge Join
- The Gather and Merge parallel nodes

Sequential Scan

Sequential Scan (Seq Scan) is the only node that is always available to the optimizer and the executor, in particular when there is no other valuable alternative. In a sequential scan, the executor will go to the beginning of the dataset on the disk – for example, the beginning of the file corresponding to a table – and will read all the data one block after the other in sequential order.

This node is, for example, always used when you ask for the contents of a table without any particular filtering clause, such as in the following example:

```
SELECT * FROM categories;
```

The **Sequential Scan** node is also used when the filtering clause is not very limiting in the query so that the end result will be to get almost the whole table contents. In such a case, the database can perform a sequential read-all operation faster, throwing away those tuples that are filtered out by the query clauses.

Index nodes

An index scan has access to the data that involves an index in order to quickly find the requested dataset. In PostgreSQL, all indexes are secondary, meaning that they live alongside the table; therefore, you will have in storage a data file for the table and one for every index you build on the table. This means that an index scan always requires two distinct accesses to the storage: one to read the disk and extract the information of where in the table the requested tuples are, and another to access the disk to seek the tuples pointed out by the index.

From this, it should be clear that PostgreSQL avoids using indexes when they are not useful, which is when the previously mentioned double storage access accounts for more disadvantages than advantages.

However, when PostgreSQL believes that accessing the data through an index could be valuable, it will produce an index node that can specialize in three different types.

Index Scan is, as the name suggests, the “classical” index access method: PostgreSQL reads the chosen index, and from that, it goes seeking the tuples, reading again from the storage.

Index-Only Scan is a particular type of **Index Scan**: if the requested data only involves columns that belong to the index, PostgreSQL is smart enough to avoid the second trip to storage since it can extract all the required information directly from the index.

The last type of index-based node you can encounter is **Bitmap Index Scan**: PostgreSQL builds a memory bitmap of where tuples that satisfy the statement clauses are, and then this bitmap is used to locate those tuples. **Bitmap Index Scan** is usually associated with **Bitmap Heap Scan**, as you will see in the examples in the following sections.

Join nodes

When PostgreSQL performs a join between two (or more) tables, it uses one out of three possible nodes. In this section, we will describe these join nodes, considering a join between two tables: an outer table (to the left of a join) and an inner one (the table on the right side of a join).

The simplest node to understand is **Nested Loop**: both tables are scanned in a sequential or indexed-based method and every tuple is checked to see whether there is a match. Essentially, the algorithm can be described by the following piece of pseudo-Java code:

```
for ( Tuple o : outerTable )
    for ( Tuple i : innerTable )
        if ( o.matches( i ) )
            appendTupleToResultDataSet( o, i );
```

As you can see from the preceding pseudo-code, **Nested Loop** is named after the nesting of the loops it performs in order to evaluate every tuple between the inner and the outer tables.

As it happens, a **Nested Loop** is not forced to perform a sequential scan on both tables, and, in fact, depending on the context, every table could be walked in a sequential or indexed-based access method. However, the core of the **Nested Loop** does not change: there will always be a nested double loop to search for matches among the tuples.

PostgreSQL chooses **Nested Loop** only if the inner table is small enough so that looping every time over it does not introduce any particular penalties.

Another way to perform a join is by using a **Hash Join** node: the inner table is mapped into a hash, which is a set of buckets containing the tuples of the table; the outer table is then walked and for every tuple extracted from the outer table, the hash is searched to see whether there is a match. The following piece of pseudo-Java code illustrates the mechanics of **Hash Join**:

```
Hash innerHash = buildHash( innerTable );
for ( Tuple o : outerTable )
    if ( innerHash.containsKey( buildHash( o ) ) )
        appendTupleToResultDataSet( o, i );
```

As you can see from the preceding example, the first step involves hashing the inner table and then walking across the outer table to see whether any of its tuples match the values in the hash map of the inner table.

The last type of join you can encounter in PostgreSQL is **Merge Join**. As the name suggests, **Merge Join** involves a step of sorting: both the tables are first sorted by the join key(s), and then they are walked sequentially. For every tuple of the outer table, all the tuples that do match in the inner table are extracted. Since both tables are sorted, a non-matching tuple indicates that it is time to move on to the next join key.

The following pseudo-Java code illustrates the algorithm of a **Merge Join**:

```
outerTable = sort( outerTable );
innerTable = sort( innerTable );
int innerIdx = 0;

for ( Tuple o : outerTable )
    for ( ; innerIdx < innerTable.length(); innerIdx++ ){
        Tuple i = innerTable[ innerIdx ];
```

```
if ( o.matches( i ) )
    appendTupleToResultSet( o, i );
else
    break;
}
```

As you can see, once the tables have been sorted, a tuple is extracted from the outer table and is compared with all the tuples within the inner table. As soon as the tuples do not match anymore, another tuple from the outer table is extracted and the inner table restarts its loop from the previous position. In other words, both tables are walked exactly once.

We will now move on to parallel nodes.

Parallel nodes

Parallel nodes are those nodes that PostgreSQL can execute to distribute the amount of work among parallel processes, therefore getting to the final result faster. It is important to note that parallel execution is not always the right choice: there is a setup time to distribute the job among parallel processes, as well as the time and resources needed to return the results of every single process. For this reason, PostgreSQL enables parallel execution of certain nodes only if the estimated parallel version will provide a benefit over sequential execution.

As a simple example, consider a case where you have a very tiny table made by only a few tuples, such as four. If you require all the table content, the resources and time spent in launching and synchronizing parallel processes will be much greater than going directly to the table and getting back the result dataset sequentially. The rule of thumb is: if the requested dataset is small enough, PostgreSQL will never choose parallel execution.

It is important to understand the fact that just because the planner produces a parallel plan, which is an execution plan made of parallel nodes, it does not mean that the executor will follow this parallelism. There could be conditions, in particular at runtime, that prevent PostgreSQL from executing a parallel plan, even if that would be the optimal choice (for instance, PostgreSQL does not have enough room to spawn the required number of parallel processes).

In the following subsection, you will learn what the main parallel nodes available are.

Gather nodes

A parallel execution plan always involves two types of **Gather** nodes: a plain **Gather** node and a **Gather Merge** node.